

Camada de Transporte

Primeira parte

Irineu Sotoma
irineu.sotoma@ufms.br

Roteiro

☐ Características

☐ Portas de Comunicação Cliente e Servidor

☐ UDP

☐ TCP

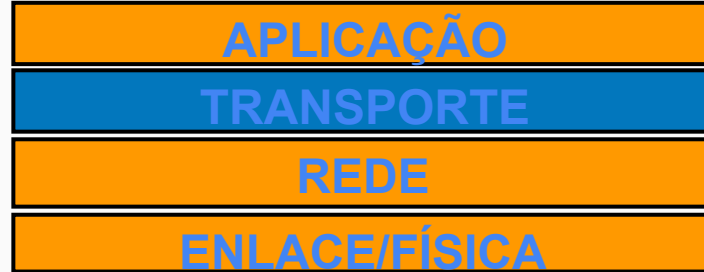
- Gerenciamento da conexão
- Princípios da transferência confiável de dados
- Controle de Erros e de Fluxo no TCP

☐ Requisitos de Aplicações

Camada de Transporte

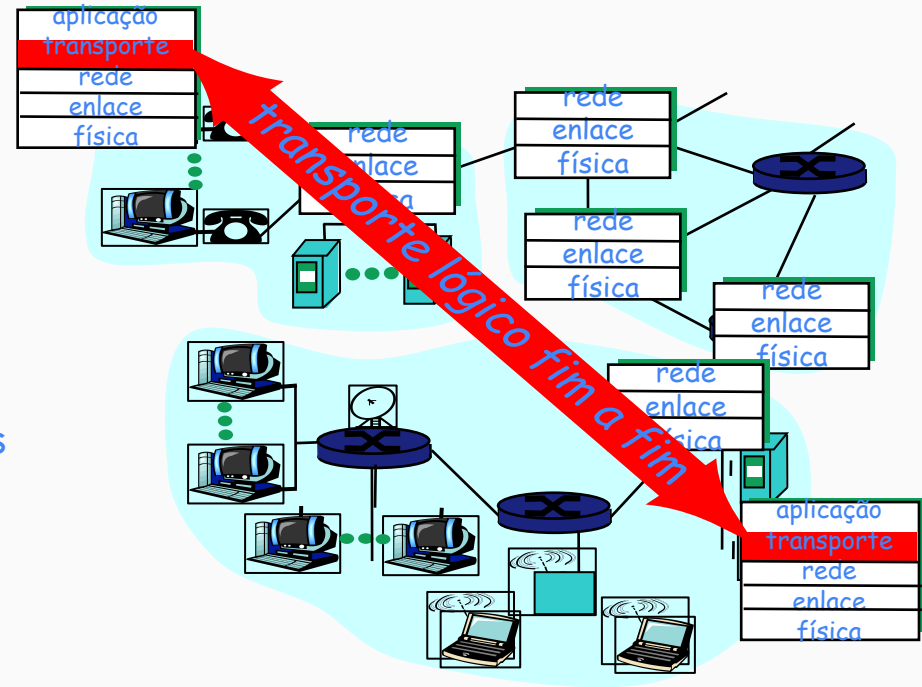
- ❑ Responsável pelo transporte dos dados do serviço de transporte

Arquitetura TCP/IP



Serviços e protocolos de transporte

- fornecem **comunicação lógica** entre processos de aplicação executando em diferentes hospedeiros
- os protocolos de transporte são executados nos sistemas finais:
 - lado transmissor: quebra as mensagens da aplicação em **segmentos**, repassa-os para a camada de rede
 - lado receptor: remonta as mensagens a partir dos segmentos, repassa-as para a camada de aplicação
- existe mais de um protocolo de transporte disponível para as aplicações
 - Internet: TCP e UDP



Camadas de Transporte x Rede

❏ *camada de rede:* comunicação lógica entre hospedeiros

❏ *camada de transporte:* comunicação lógica entre os processos

- depende de, estende serviços da camada de rede

Analogia doméstica:

12 crianças na casa de Ana enviando cartas para 12 crianças na casa de Bill

- hospedeiros = casas
- processos = crianças
- mensagens da apl. = cartas nos envelopes
- protocolo de transporte = Ana e Bill multiplexam enviando as cartas via serviço postal. Ana e Bill que demultiplexam para suas crianças
- protocolo da camada de rede = serviço postal

Características

- ❑ Comunicação fim-a-fim entre aplicações
 - Canal virtual de comunicação entre aplicações em computadores distintos
- ❑ Rede básica (IP) opera pelo melhor esforço (*best-effort*)
 - Pacotes perdidos
 - não pede mensagens novamente
 - entrega cópias duplicadas de determinada mensagem
 - limita as mensagens a algum tamanho finito
 - entrega mensagens após um atraso arbitrariamente longo

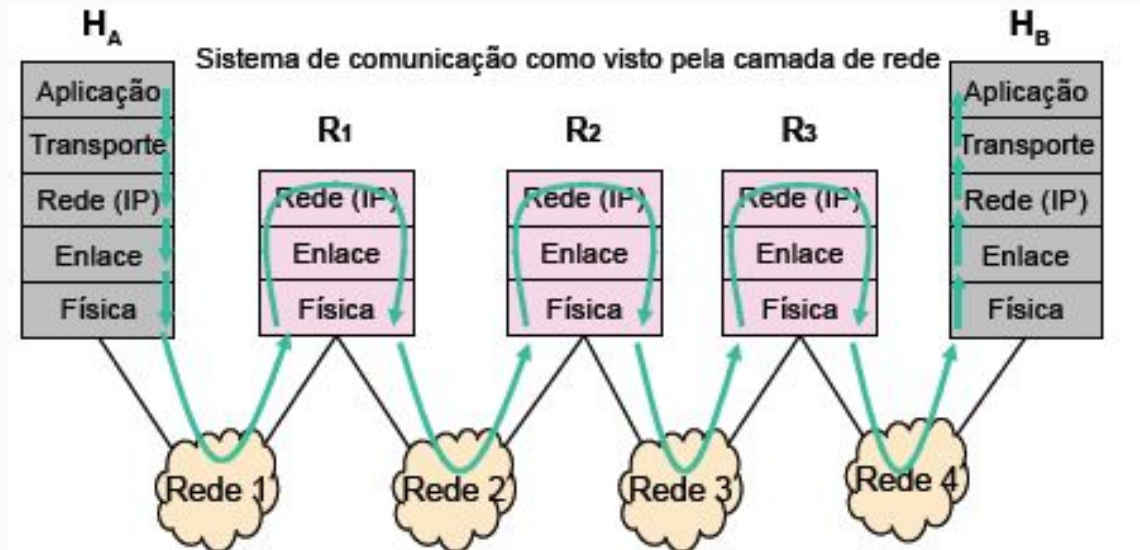
Características

Serviços fim-a-fim usuais

- garante entrega da mensagem
- entrega mensagens na mesma ordem em que foram enviadas
- entrega no máximo uma cópia de cada mensagem
- aceita mensagens arbitrariamente grandes
- detecção e correção de erro fim-a-fim
- aceita sincronismo entre fonte e destino
- permite que o receptor controle o fluxo do emissor
- admite múltiplos processos de aplicação em cada host

Características: Serviço fim-a-fim

- ❑ Canais são implementados em software, já que o subsistema de rede não provê facilidades
- ❑ Mensagens TCP e UDP são encapsuladas em datagramas (pacotes) IP



Características: Serviço fim-a-fim

- ❑ Canais são implementados em software, já que o subsistema de rede não provê facilidades
- ❑ Mensagens TCP e UDP são encapsuladas em datagramas (pacotes) IP



Características

❑ Mecanismos de identificação de processos

❑ O nível de transporte oferece:

- serviços sem conexão (datagramas)

Nenhuma confiabilidade, mas com baixo overhead

Ex: UDP

- serviços orientados à conexão (circuitos virtuais)

Controle do fluxo de dados fim-a-fim

Controle de erro □ detecção e correção de erro fim-a-fim

Controle de sequência (sequenciação)

Divisão de mensagens em segmentos

Ex: TCP

Roteiro

❑ Características

❑ Portas de Comunicação Cliente e Servidor

- Multiplexação e Demultiplexação

❑ UDP

❑ TCP

- Gerenciamento da conexão
- Princípios da transferência confiável de dados
- Controle de Erros e de Fluxo no TCP

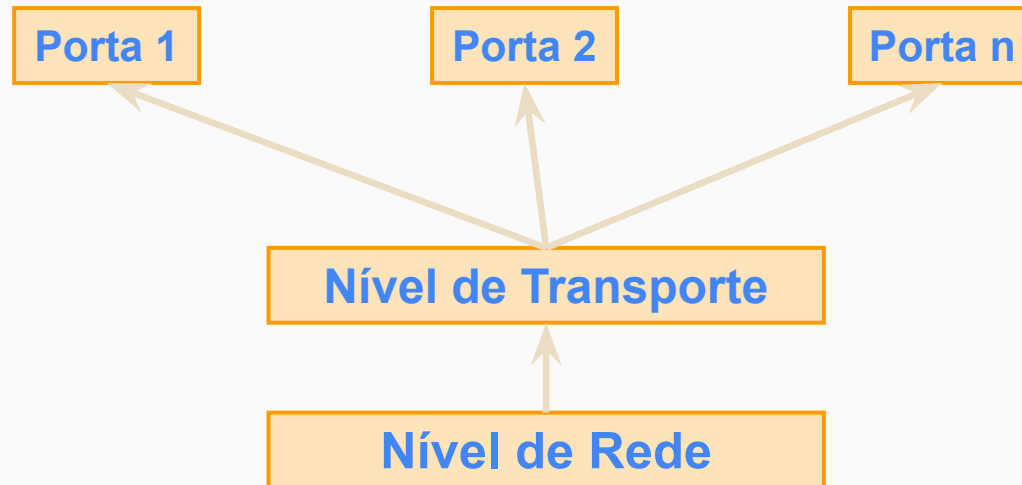
❑ Requisitos de Aplicações

Portas

- ❑ Identificam os processos origem e destino
- ❑ Viabilizam a comunicação fim-a-fim
- ❑ Permitem envio e recepção de datagramas de forma independente

Multiplexação e Demultiplexação

- ❑ Através das **portas**, o protocolo de transporte identifica para qual processo ele tem que entregar as informações na camada de aplicação!



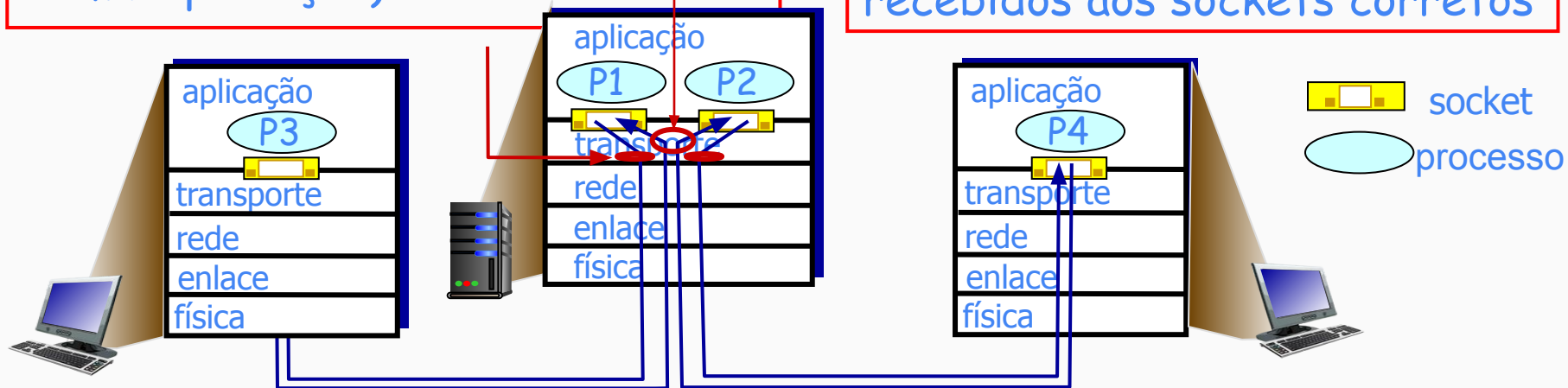
Multiplexação/demultiplexação

Multiplexação no transmissor:

reúne dados de muitos sockets, adiciona o cabeçalho de transporte (usado posteriormente para a demultiplexação)

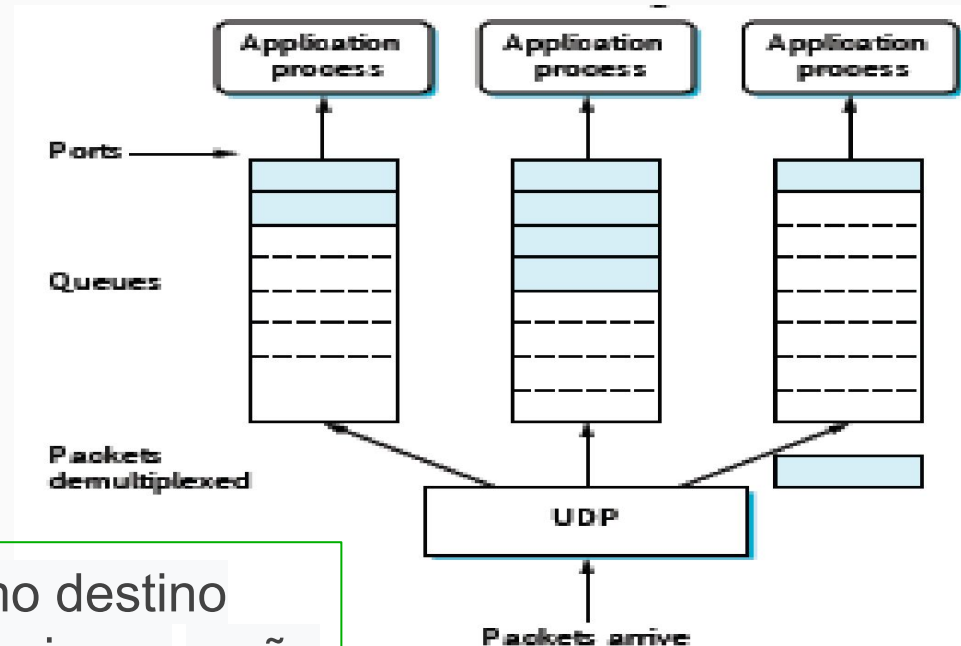
Demultiplexação no receptor:

Usa info do cabeçalho de transporte para entregar os segmentos recebidos aos sockets corretos



Multiplexação e Demultiplexação

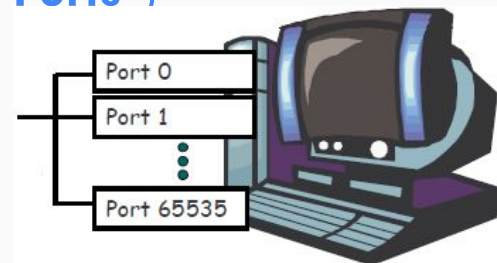
- Portas
- SO oferece interface de acesso
- Pressupõe *buffers* de recepção e transmissão



OBS: Datagramas UDP com o mesmo destino (IP, porta), mas vindos de diferentes origens serão direcionados para o mesmo soquete no host receptor

Portas

- ❑ Portas são números inteiros de 16 bits
- ❑ Número das portas:
 - 0-1023: reservado, deve ser root (“Well Known Ports”)
 - 1024 – 49151: pode ser registrado (“Registered Ports”)
 - 49152 – 65535: Geral (“Dynamic and/or Private Ports”)
- ❑ Alguns dos serviços em well known ports
 - DNS 53/tcp e 53/udp
 - HTTP 80/tcp
 - HTTPS 443/tcp
 - SMTP 25/tcp
 - POP3 110/tcp
- ❑ Alguns dos serviços em registered ports, em que cada servidor pode operar em diferentes portas configuráveis
 - MySQL 3306/tcp
 - Docker API 2375/tcp
- ❑ Dynamic and/or Private ports
 - Sem registro: cada empresa usa para suas próprias aplicações; não há padronização ou necessidade de interoperabilidade global



Portas

❑ RFC 1700 (1994)

- Definiu inicialmente uma tabela estática com os serviços e portas

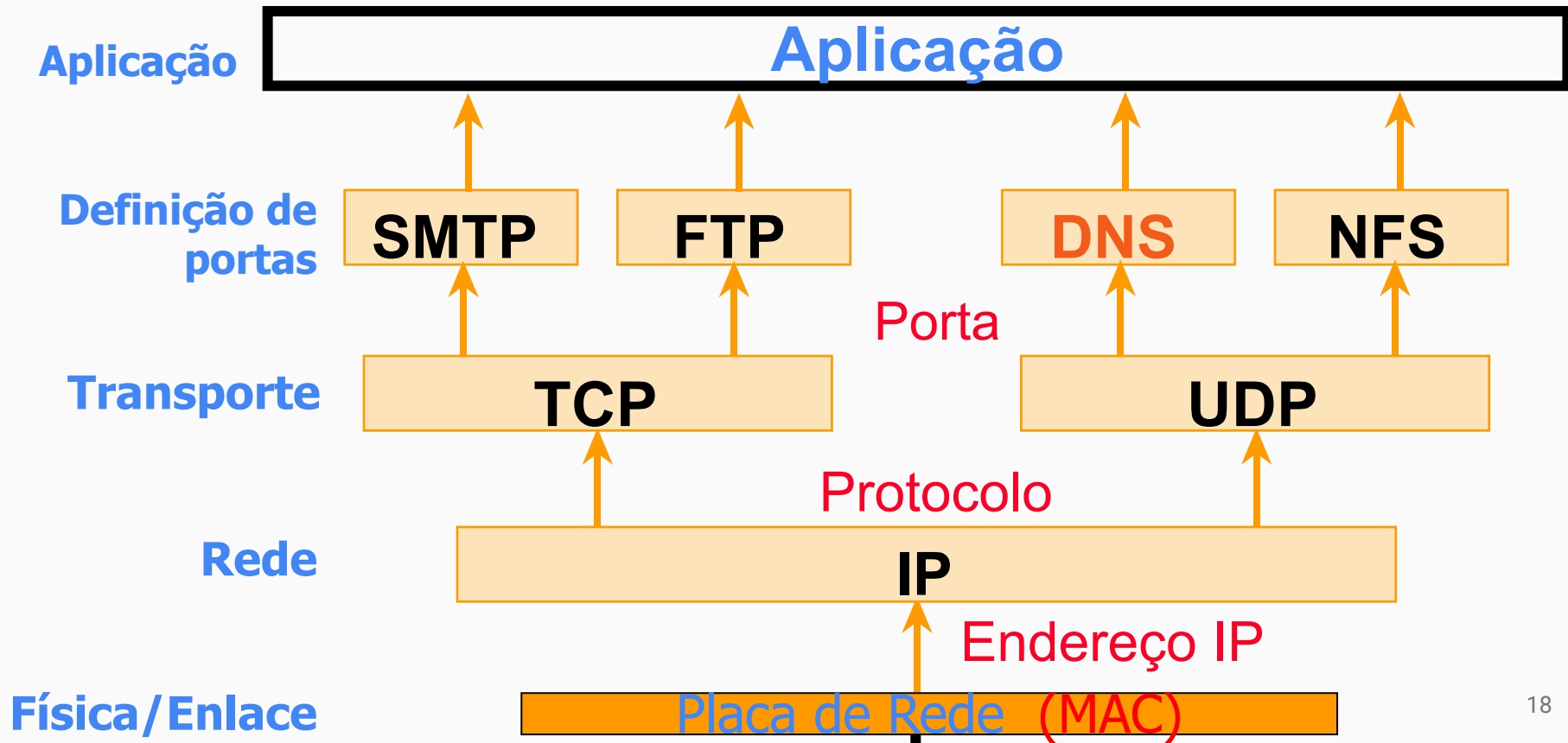
❑ Entre 1994 e 2002

- Não tinha como publicar uma nova RFC a cada atualização da RFC 1700, então decidiu-se que o ISI (Information Sciences Institute) e a IANA (Internet Assigned Numbers Authority) iriam disponibilizar atualizações online.

❑ RFC 3232 (2002):

- Oficializou o que ocorria desde 1994.
- Tornou a RFC 1700 obsoleta e passou a responsabilidade apenas para a IANA online:
<https://www.iana.org/assignments/service-names-port-numbers>

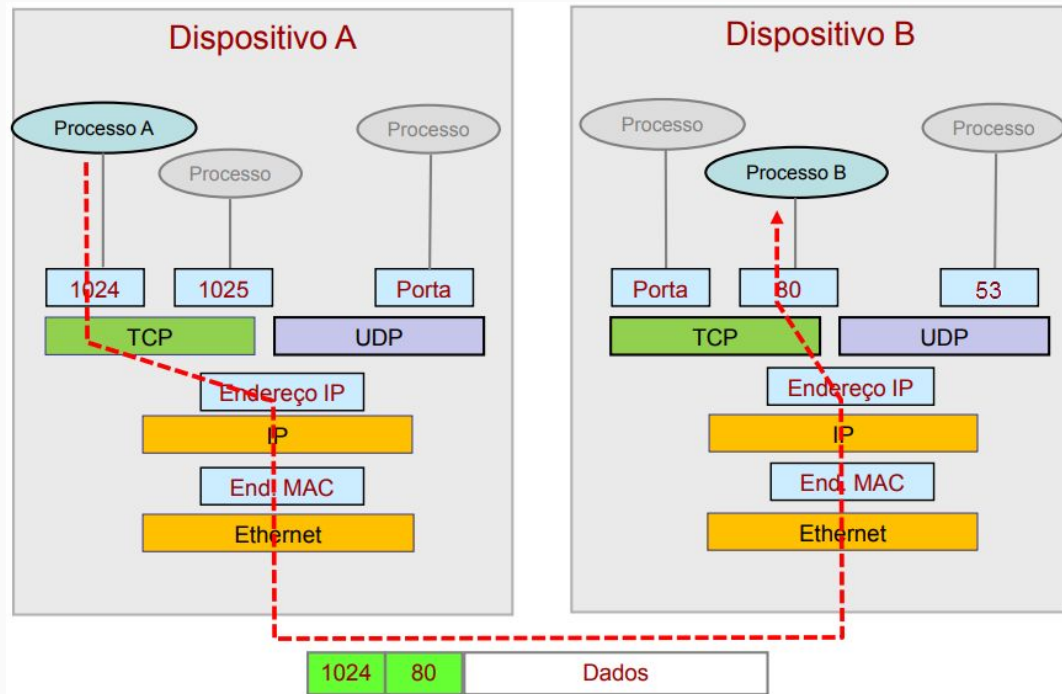
Multiplexação e Demultiplexação



Multiplexação e Demultiplexação

Soquete TCP identificado por 4-tupla:

- IP, Porta origem
- IP, Porta destino



Roteiro

❑ Características

❑ Portas de Comunicação Cliente e Servidor

❑ UDP

❑ TCP

- Gerenciamento da conexão
- Princípios da transferência confiável de dados
- Controle de Erros e de Fluxo no TCP

❑ Requisitos de Aplicações

Protocolos

❑ UDP (User Datagram Protocol):

- Serviço não orientado à conexão
- Simples, Baixo overhead
- Não confiável
 - Detecção de erro opcional
 - Sem controle de sequência , sem controle de fluxo

❑ TCP (Transmission Control Protocol):

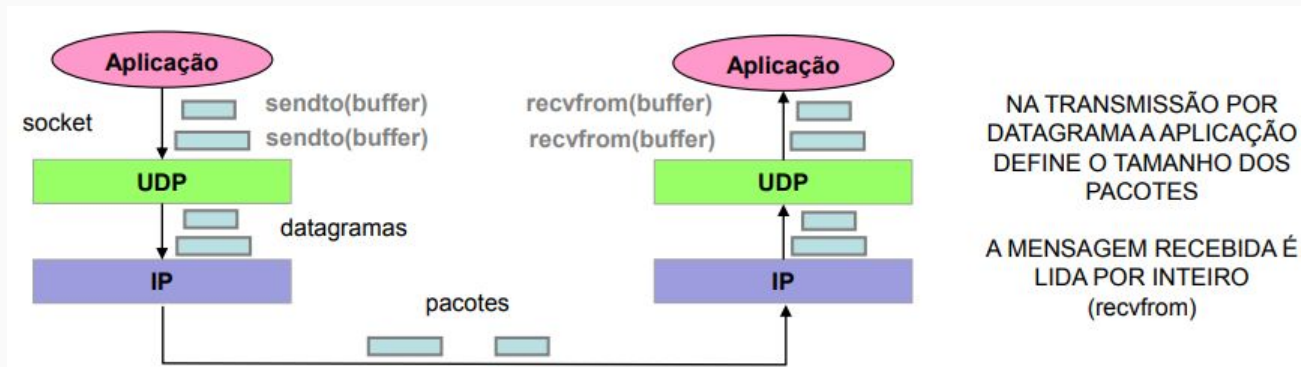
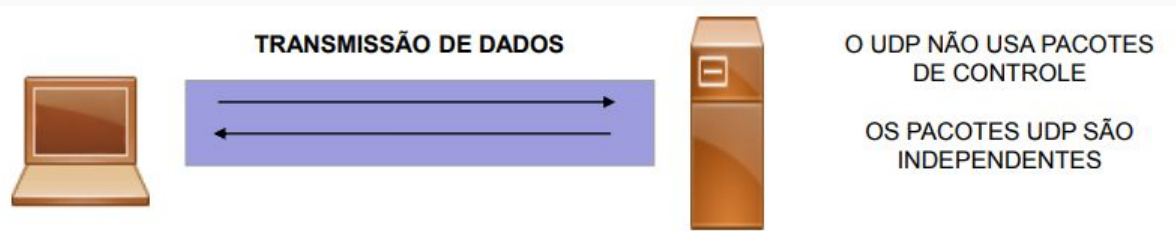
- Serviço orientado a conexão
- Confiável
- Detecção e correção de erros
- Controle de fluxo fim-a-fim, controle de congestionamento

❑ Outros (SCTP, QUIC (na camada de aplicação))

UDP: User Datagram Protocol [RFC 768]

- ❑ Protocolo de transporte da Internet mínimo, “sem gorduras”,
- ❑ Serviço “melhor esforço”, segmentos UDP podem ser:
 - perdidos
 - entregues à aplicação fora de ordem
- ❑ sem conexão:
 - não há saudação inicial entre o remetente e o receptor UDP
 - tratamento independente para cada segmento UDP
- ❑ Uso do UDP:
 - aplicações de streaming multimídia (tolerante a perdas, sensível a taxas)
 - DNS
 - SNMP
 - HTTP/3
- ❑ Transferência confiável sobre UDP:
 - adiciona confiabilidade na camada de aplicação
 - recuperação de erros específica da aplicação

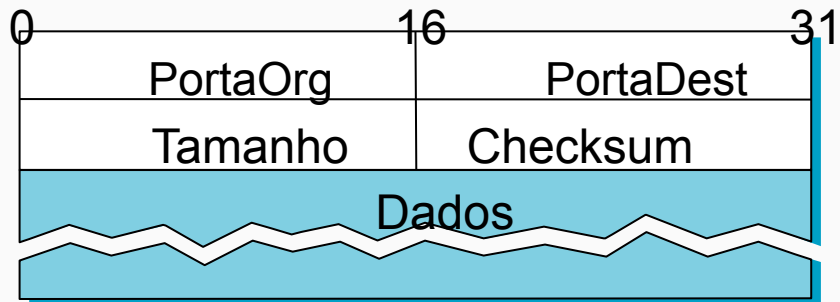
UDP



UDP

Formato do Datagrama UDP

- **Source/Destination Port:**
 - Portas usadas pelos processos
- **Length:**
 - Tamanho do datagrama (octetos), incluindo o cabeçalho
- **Checksum:**
 - Verificação de erro no datagrama (opcional no IPv4 e obrigatório no IPv6).
 - Pseudo-cabeçalho IP + cabeçalho UDP + dados
- **Data:**
 - Dados do usuário



UDP: Pseudo-cabeçalho IP (exemplo com IPV4)

Campo	Tamanho (em bits)	Descrição Detalhada
Endereço IP de Origem	32	Extraído diretamente do cabeçalho IP, este campo contém o endereço do host que enviou o datagrama UDP. A inclusão deste endereço no cálculo do checksum ajuda a garantir que o pacote não foi originado de uma fonte inesperada.
Endereço IP de Destino	32	Também obtido do cabeçalho IP, este campo especifica o endereço do host de destino. Sua presença no cálculo do checksum é crucial para verificar se o datagrama foi entregue à máquina correta, e não a outra que possa ter o mesmo endereço na camada de enlace, por exemplo.
Campo de Zeros	8	Este campo de um byte é preenchido com zeros. Sua principal função é preencher o pseudo-cabeçalho para que ele se alinhe em um limite de 16 bits, facilitando o cálculo do checksum.
Protocolo	8	Este campo contém o número do protocolo da camada de transporte que está sendo utilizado. Para o UDP, este valor é sempre 17. A inclusão deste número confirma que o pacote que a camada IP está entregando para a camada de transporte é, de fato, um datagrama UDP, e não de outro protocolo como o TCP (que tem o número 6).
Comprimento UDP	16	Este campo especifica o comprimento total do datagrama UDP, incluindo o cabeçalho UDP e os dados do usuário, em bytes. É importante notar que este campo é duplicado do próprio cabeçalho UDP e sua inclusão no pseudo-cabeçalho serve como uma verificação cruzada para garantir que o tamanho do datagrama não foi corrompido durante a transmissão.

Soma de Verificação (*checksum*) UDP

Objetivo: detectar “erros” (ex.: bits trocados) no segmento transmitido

Transmissor:

- ❑ trata conteúdo do segmento como sequência de inteiros de 16-bits
- ❑ checksum: soma (adição usando complemento de 1) do conteúdo do segmento
- ❑ transmissor coloca *complemento do valor da soma* no campo *checksum* do UDP

Receptor:

- ❑ calcula *checksum* do segmento recebido
- ❑ verifica se o *checksum* calculado é 0xFFFF:
 - NÃO - erro detectado
 - SIM - nenhum erro detectado. *Mas ainda pode ter erros? Veja depois*

Soma de Verificação (checksum) UDP

Transmissor:

O processo de cálculo do checksum UDP funciona da seguinte maneira:
O remetente cria o pseudo-cabeçalho IP com as informações mencionadas anteriormente.

Em seguida, concatena o pseudo-cabeçalho, o cabeçalho UDP (com o campo de checksum zerado temporariamente) e os dados do usuário.

O checksum é então calculado sobre toda essa sequência de bytes.

O complemento de 1 do checksum resultante é inserido no campo "Checksum" do cabeçalho UDP.

O datagrama UDP, encapsulado em um pacote IP, é finalmente enviado pela rede.

Soma de Verificação (checksum) UDP

Receptor:

Ao receber o pacote, o destinatário executa o mesmo procedimento:

Monta o pseudo-cabeçalho a partir do cabeçalho IP recebido;

Soma o pseudo-cabeçalho, o cabeçalho UDP (incluindo o checksum) e os dados.

Se essa soma calculada for igual a 0xFFFF, o datagrama é considerado íntegro e corretamente endereçado. Caso contrário, o pacote é descartado.

Justificativa Normativa (Por que FFFF?)

A **RFC 768** e a **RFC 1071** escolhem essa inversão porque, matematicamente, em complemento de um:

$A + \sim A = -0$ (ou seja, todos os bits em 1)}

Isso permite que o receptor não precise fazer subtrações ou comparações complexas; ele apenas "acumula" os bits. Se no final o registrador não estiver "cheio" de bits 1, o pacote é descartado.

Exemplo de Cálculo Checksum de Remetente UDP

Passo A: Montagem dos Dados (16-bit words)

Dados os parâmetros: IP Origem 192.168.0.1, IP Destino 192.168.0.2, Portas origem 50000 e destino 55000, e dados 1234 (número inteiro).

I. Pseudo-Cabeçalho (conforme RFC 768 de 1980)

1. IP Origem (MSB): C0A8 (192.168)
2. IP Origem (LSB): 0001 (0.1)
3. IP Destino (MSB): C0A8 (192.168)
4. IP Destino (LSB): 0002 (0.2)
5. Zeros + Protocolo: 0011 (Onde 11 hex = 17 decimal, o código do UDP)
6. UDP Length: 000A (10 bytes: 8 do cabeçalho + 2 de dados)

II. Cabeçalho UDP Real (conforme RFC 768)

7. Source Port: C350 (50000)
8. Dest Port: D6D8 (55000)
9. Length: 000A (10 bytes)
10. Checksum: 0000 (Zerado para o cálculo, conforme RFC 1071, de 1988)

III. Carga Útil (Payload)

11. Dados: 04D2 (Valor decimal 1234 em hex)

Exemplo de Cálculo Checksum de Remetente UDP

Passo B: Aritmética de Complemento de Um (conforme RFC 1071, 1988)

A **RFC 1071** dita que somamos as palavras e aplicamos o *end-around carry* a cada overflow de 16 bits.

1. $C0A8 + 0001 = C0A9$
2. $C0A9 + C0A8 = 18151 \rightarrow \text{Carry: } 8151 + 1 = 8152$
3. $8152 + 0002 = 8154$
4. $8154 + 0011 = 8165$
5. $8165 + 000A = 816F$
6. $816F + C350 = 144BF \rightarrow \text{Carry: } 44BF + 1 = 44C0$
7. $44C0 + D6D8 = 11B98 \rightarrow \text{Carry: } 1B98 + 1 = 1B99$
8. $1B99 + 000A = 1BA3$
9. $1BA3 + 04D2 = 2075$ (Soma Acumulada Final)

Passo C: Inversão Final

O Checksum é o complemento de um da soma:

- Soma: **0010 0000 0111 0101 (2075)**
- Checksum transmitido (NOT): **1101 1111 1000 1010 (DF8A)**

Exemplo de Cálculo Checksum de Destinatário UDP

O receptor recebe o pacote e, conforme a **RFC 1071 (Seção 4.1)**, deve realizar a soma de todas as palavras de 16 bits, incluindo o Checksum que o remetente inseriu.

A Soma de Verificação:

O destinatário reconstrói o pseudo-cabeçalho (usando os IPs que vieram no cabeçalho IP do pacote recebido) e soma tudo:

1. Soma de todos os campos (exceto checksum): 2075
2. Adição do Checksum recebido: DF8A
3. $2075 + DF8A = FFFF$

Resultado:

O valor FFFF (todos os bits em 1) prova que a soma é válida. Se houvesse um erro e o resultado fosse 1FFFF, o receptor aplicaria o *end-around carry*:

- $FFFF + 1 = 10000 \rightarrow 0000 + 1 = 0001$.
- Como $0001 \neq FFFF$, o pacote seria sumariamente descartado.

Cálculo Checksum no IPV6

- **Obrigatoriedade:** No IPv4, um checksum de 0000 significa "nenhum checksum calculado". No IPv6, isso é proibido. Se o resultado do cálculo for 0000, ele **deve** ser convertido para FFFF para ser enviado (**RFC 2460, Seção 8.1**).
- **Campo de Comprimento:** No pseudo-cabeçalho IPv6, o campo *Payload Length* tem **32 bits** (para suportar *Jumbograms* da **RFC 2675**), enquanto no cabeçalho UDP real ele continua tendo 16 bits.
- **Proteção de Endereço:** Como o roteamento IPv6 pode envolver cabeçalhos de extensão complexos, o Checksum no UDP é a única garantia de que o endereço de destino não foi alterado erroneamente por um roteador intermediário (já que o cabeçalho IPv6 não tem Checksum para proteger a si mesmo).

Resumo: UDP

❑ protocolo “sem frescura”:

- segmentos podem ser perdidos, entregues fora de ordem
- serviço de melhor esforço: “envie e espere pelo melhor”

❑ O UDP tem seus pontos positivos:

- nenhuma conexão/handshaking é necessário (sem gastar um RTT)
- pode funcionar quando o serviço de rede está comprometido
- ajuda na confiabilidade (soma de verificação)

❑ construir funcionalidade adicional sobre UDP na camada de aplicação (por exemplo, HTTP/3)

Roteiro

❑ Características

❑ Portas de Comunicação Cliente e Servidor

❑ UDP

❑ TCP

- Gerenciamento da conexão
- Princípios da transferência confiável de dados
- Controle de Erros e de Fluxo no TCP

❑ Requisitos de Aplicações

TCP

- ❏ Características:
 - Ponto a ponto: um transmissor, um receptor
 - Baseado em Stream:
 - Sequência de bytes não estruturada
 - Conexão com Circuito Virtual (Orientado à Conexão):
 - Estabelecimento, transmissão e encerramento de conexão
 - Confiável (detecção e correção de erros)
 - Buffers de envio e recebimento:
 - Buffer de retransmissão com controle automático de envio
 - Fluxo de dados *full duplex* (bidirecional)
 - Controle de fluxo por janela deslizante:
 - evita que emissor ultrapasse a capacidade do receptor
 - Controle de congestionamento:
 - evita que o emissor inunde a rede

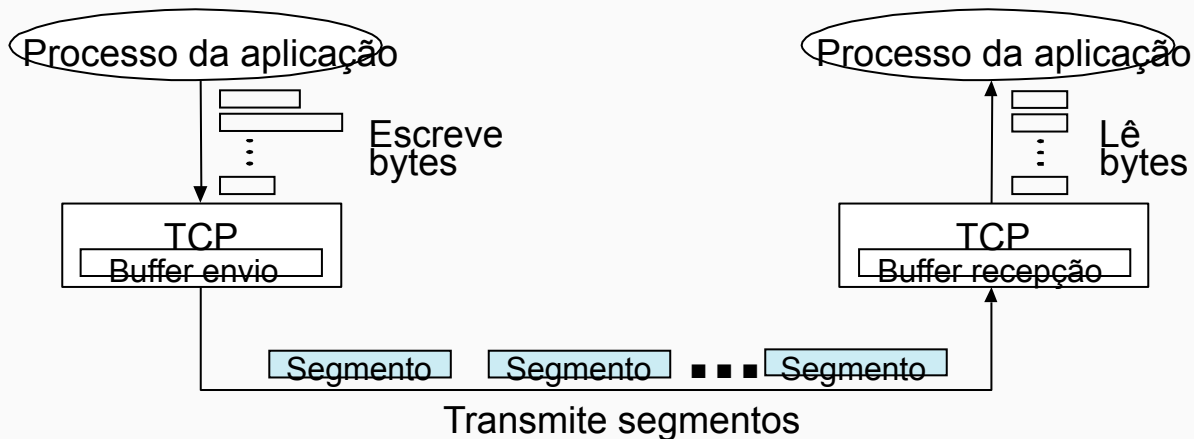
TCP

Segmentos:

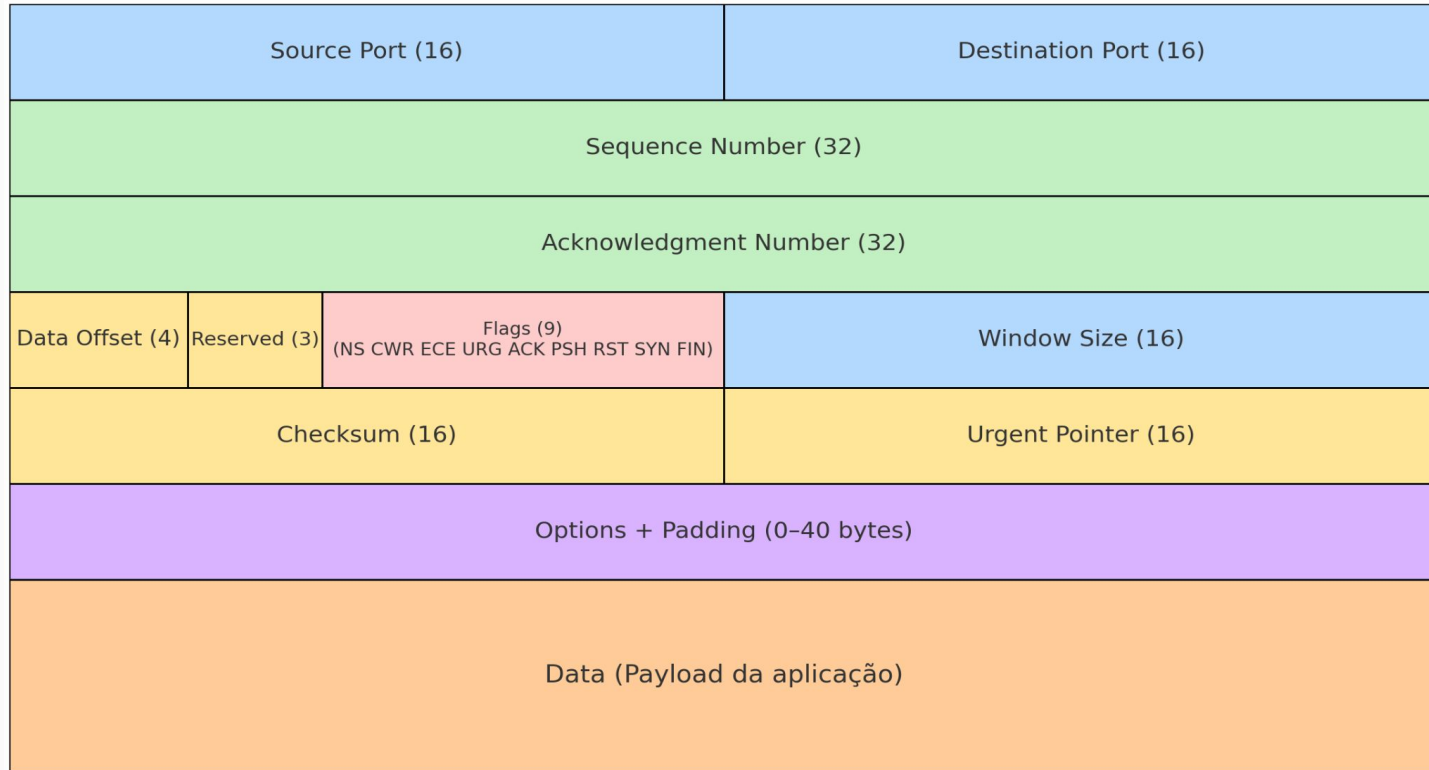
- TCP manipula o *stream* de dados como uma sequência de *bytes* divididos em segmentos
- Cada segmento é encapsulado dentro de um datagrama IP

Números de Sequência:

- Segmentos no *stream* de dados são numerados sequencialmente;



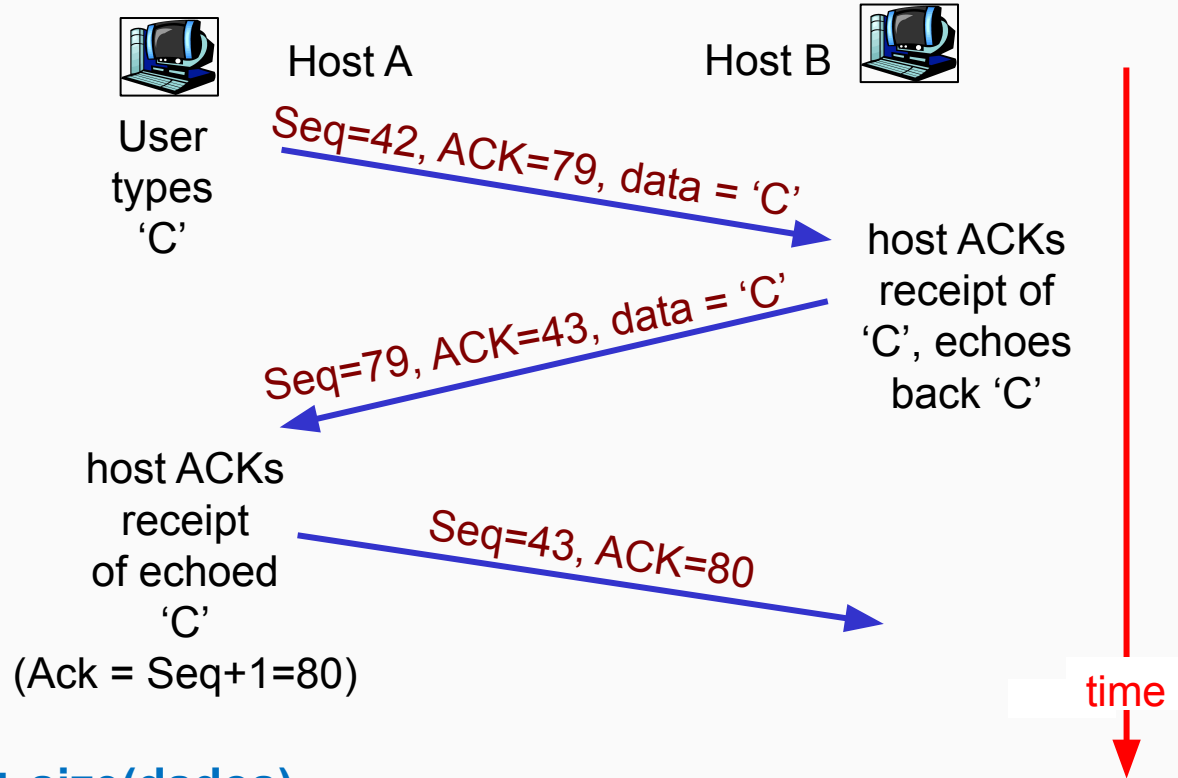
TCP: Cabeçalho + Dados



TCP (Campos do Segment)

- **Source/Destination Port:**
 - Identificam os processos origem e destino da conexão
- **Sequence Number:**
 - Posição/Número do segmento dentro do stream de dados
- **Ack Number:**
 - Confirmação (ACK): número do próximo byte que o receptor espera receber
 - Pacote que vai em um sentido confirma último pacote recebido no sentido contrário (*piggybacking*)
- **Data offset:**
 - Tamanho do cabeçalho (unidade: 4 octetos, 32 bits, 4 bytes). Indica onde começam os dados, logo após o cabeçalho.

TCP



$$\text{Ack} = \text{Seq} + \text{size}(\text{dados})$$

TCP (Campos do Segment)

● Code Bits:

São 9 bits atualmente (2025), sendo que os 3 mais recentes são:

- NS (RFC 3540 - 2003): ECN Concealment.
Tentativa de validação de ECN (ECN Nonce), mas foi abandonado (RFC 8311 - 2018). Não deve ser utilizado em conexões novas.
- CWR (RFC 3168 - 2001): Congestion Window Reduced.
É enviado pelo transmissor depois de receber um ACK com **ECE=1**.
- ECE (RFC 3168 - 2001): ECN Echo.

Usado para **ecoar** para o transmissor que o roteador marcou o pacote com **CE (Congestion Experienced)** no cabeçalho IP. Faz parte do mecanismo **ECN (Explicit Congestion Notification)**, que permite sinalizar congestionamento **sem descartar pacotes**. No handshake: se o cliente anuncia **ECE=1** e **CWR=1** no SYN, indica suporte a ECN. Durante a sessão: receptor marca **ECE=1** nos ACKs para avisar o transmissor sobre congestionamento.

TCP (Campos do Segmento)

- **Code Bits:**

Os 6 bits mais à direita foram definidos pela RFC 793 em 1981:

- URG: Campo Urgent Pointer válido. RFC 6093 (2011) desaconselha a utilização.
- ACK: Confirmação do pedido de conexão.
- PSH: Segmento requer entrega imediata (push). Utilizado em aplicações interativas (Telnet e SSH). Mas essa entrega imediata não é obrigatória.
- RST: Reseta a conexão. Término devido a erro, por exemplo, pacote chega em porta sem processo escutando, ou uma das pontas aborta de forma abrupta.
- SYN: Estabelecimento de conexão. Sincroniza números de sequência.
- FIN: Origem finalizou seu stream de bytes. Encerramento da Conexão.

TCP (Campos do Segment)

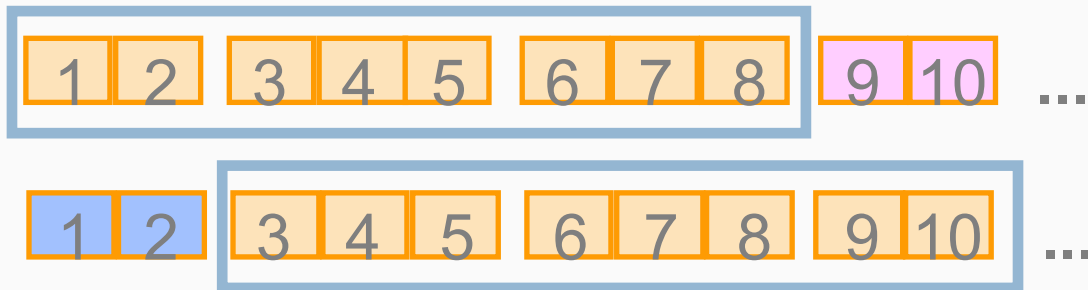
- **Window Size:**

- Especifica o tamanho da janela
 - Possibilita controle de fluxo
 - Evita a sobrescrita de *buffers*
 - Informa o tamanho da janela disponível no receptor
 - Transmissor não pode enviar dados além do que o receptor pode receber
- Exemplo:
ACK=2000, Window Size=3000
Receptor aceita bytes **2000 até 4999**.
Emissor só pode mandar dados nesse intervalo.

TCP

❑ Sliding Window:

- Permite o transmissor enviar múltiplos segmentos antes de receber reconhecimento
- Eficiência de transmissão e controle de fluxo dinâmico
 - É informado junto com a confirmação do número de bytes que o receptor tem capacidade de receber.



TCP (Campos do Segmento)

- **Checksum:**

- Verificação de erro no segmento: o cálculo é similar ao do UDP

- **Urgent Pointer:**

- Posição dos dados urgentes no segmento (se existirem). Há inconsistência no uso por APIs e Middleboxes e a RFC 6093 (2011) desaconselha para novas aplicações.

TCP (Campos do Segmento)

- **Options (Há várias outras além das seguintes):**
 - **Tamanho de Segmento Máximo (MSS) (RFCs 793 e 879)**
 - Pacote de pesquisa pelo MSS
Cada lado anuncia seu **MSS** no **SYN**.
O emissor deve respeitar o menor MSS anunciado pelo par
 - Visa evitar fragmentação IP
 - Se não anunciado, assume-se 536 bytes no IPv4 e 1220 bytes no IPv6.
 - MSS é **negociado apenas no SYN**, não muda durante a conexão.
 - Exemplo:
Ethernet comum: MTU = 1500 bytes.
Cabeçalho IP (20 bytes) + cabeçalho TCP (20 bytes).
MSS típico = **1460 bytes**.

TCP (Campos do Segment)

- **Options (Há várias outras além das seguintes):**
 - Window Scale (Fator de escala do tamanho da janela) (RFC 7323)
 - O campo **Window Size** tem só **16 bits**, máximo 64 KB.
 - Em redes de **alta largura de banda e latência alta**, 64 KB é insuficiente → limitaria a taxa de envio.
 - Permite janelas maiores que 64KB
 - Janela efetiva = Window Size $\ll$ Shift
 - Exemplo
Receptor anuncia **Window Size = 65.535** e **Shift = 7**.
Janela real = **$65.535 \times 2^7 = 8.388.480$ bytes (≈ 8 MB)**.

TCP (Campos do Segment)

- **Options (Há várias outras além das seguintes):**
 - *Timestamps (RFC 7323)*
 - Permite reconhecer pacotes que tenham se extraviado pela rede, duplicatas antigas, durante conexões muito rápidas
 - Permite um RTT fino
 - Encontradas em quase todas as implementações modernas
 - Essenciais para garantir desempenho nas redes de alta capacidade

TCP (Campos do Segment)

- **Padding:**

- O cabeçalho TCP varia de 20 bytes (sem opções) ao máximo de 60 bytes (com 40 bytes de opções).
- O campo **Data Offset (4 bits)** indica o tamanho real do cabeçalho em *palavras de 32 bits (4 bytes)*.
- Uma vez que as opções podem ter tamanhos diferentes (2, 3, 20, ... bytes), o final do cabeçalho pode **não terminar em um múltiplo de 32 bits (4 bytes)**.
- Para garantir o alinhamento, insere-se **bytes de preenchimento com valor zero** (padding) após as opções.

TCP (Campos do Segment)

- **Exemplo de Padding:**
 - Total de opções = **7 bytes**.
 - Cabeçalho base (20 bytes) + opções (7 bytes) = **27 bytes**.
 - Para alinhar a 32 bits → precisa completar até **28 bytes** (múltiplo de 4).
 - Assim, 1 byte de **Padding (0x00)** é adicionado.
 - **Data Offset** nesse caso = $28 \div 4 = 7$.